Práctica 2 - Implementación del Analizador Sintáctico Descendente (Parser LL)

1. Introducción

En esta práctica se implementará el **analizador sintáctico (parser)** utilizando el modelo de análisis descendente basado en tabla LL.

El análisis sintáctico es la segunda fase del proceso de compilación, cuya función es verificar que la secuencia de tokens generada por el analizador léxico tenga una estructura válida de acuerdo con una gramática.

2. Objetivos

Al finalizar esta práctica, el estudiante será capaz de:

- Comprender el papel del análisis sintáctico descendente.
- Implementar el algoritmo general de un parser LL basado en tabla.
- Utilizar una pila para simular el proceso de análisis sintáctico.
- Aplicar una tabla LL para tomar decisiones durante el análisis.
- Identificar cadenas válidas e inválidas según una gramática.
- Identificar errores sintácticos en una entrada.

3. Implementación

La elección del lenguaje de programación en el que deberá implementarse el analizador sintáctico es libre, pudiendo ser Java, Python, Go, o cualquier otro que sea más cómodo para cada estudiante (de preferencia el mismo empleado en la Práctica 1).

En esta práctica se implementará un parser manual, sin utilizar generadores automáticos de parsers, lo cual permite comprender con mayor claridad cómo funciona el análisis sintáctico descendente.

El parser debe:

1. Recibir como entrada una lista de tokens, de preferencia proporcionada como arreglo.
2. Inicializar una pila con [$, SimboloInicial].
3. Procesar la entrada utilizando el algoritmo LL con tabla.

4. Determinar si la cadena es válida o inválida.
5. Reportar errores sintácticos cuando corresponda.

Esto implica implementar el siguiente algoritmo:

```
mientras la pila no esté vacía:
  X = tope de la pila
  a = token actual

  si X es terminal:
    si X == a:
      pop()
      avanzar entrada
    si no:
      error

  si X es no terminal:
    consultar tabla[X, a]
    aplicar producción:
      pop()
      push (lado derecho en orden inverso)

  si X == epsilon:
    pop()
```

Nota:

*Es posible implementar un parser equivalente utilizando el enfoque de **parser recursivo descendente**. En este caso, la lógica del algoritmo LL se traduce directamente a funciones en código que tienen las siguientes consideraciones:*

- *Cada símbolo no terminal de la gramática se implementa como una función. Ejemplo: $E()$, $T()$, $F()$.*
- *Cuando un no terminal tiene múltiples producciones, se selecciona cuál usar en función del token actual (lookahead). Esto equivale a consultar la tabla LL, pero de forma explícita en el código.*
- *Cuando una producción deriva en ε, la función simplemente no ejecuta ninguna acción adicional y regresa.*
- *Los símbolos terminales se validan mediante una función $match()$ que verifica que el token actual es el esperado y consume el token (avanza en la entrada).*

→ *Si decides implementar esta variante de parser, asegúrate de respetar la estructura de la gramática proporcionada, mantén una función por cada no terminal y valida cuidadosamente el consumo de los tokens.*

4. Ejemplos de entrada

Usar **solamente** las siguientes gramáticas y tablas LL para validar la correctitud de la implementación realizada.

Ejemplo 1.
Gramática:

```
E → T E'
E' → + T E' | ε
T → num
```

Tabla LL:

	num	+	$
E	E → T E'		
E'		E' → + T E'	E' → ε
T	T → num		

Ejemplo tabla LL en código:

```
tabla = {
  "E": {
    "num": ["T", "E'"]
  },
  "E'": {
    "+": ["+", "T", "E'"],
     "$": ["ε"]
  },
  "T": {
    "num": ["num"]
  }
}
```

Entrada válida:

```
num + num $
```

Salida esperada:

```
Pila: ['$', 'E'], Entrada: ['num', '+', 'num', '$']
Aplicar: E → ['T', "E'"]


Pila: ['$', "E'", 'T'], Entrada: ['num', '+', 'num', '$']
Aplicar: T → ['num']


Pila: ['$', "E'", 'num'], Entrada: ['num', '+', 'num', '$']
Match: num
Pila: ['$', "E'"], Entrada: ['+', 'num', '$']
Aplicar: E' → ['+', 'T', "E'"]


Pila: ['$', "E'", 'T', '+'], Entrada: ['+', 'num', '$']
Match: +
Pila: ['$', "E'", 'T'], Entrada: ['num', '$']
Aplicar: T → ['num']


Pila: ['$', "E'", 'num'], Entrada: ['num', '$']
Match: num
Pila: ['$', "E'"], Entrada: ['$']
Aplicar: E' → ['ε']


Pila: ['$'], Entrada: ['$']
Match: $
```

Nota: El formato exacto de impresión puede variar, pero debe ser claro y consistente.

Ejemplo 2.

Gramática:

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → (E) | num

Tabla LL:

No terminal	num	+	*	(	)	$
E	E → T E'			E → T E'		
E'		E' → + T E'			E' → ε	E' → ε
T	T → F T'			T → F T'		
T'		T' → ε	T' → * F T'		T' → ε	T' → ε
F	F → num			F → (E)		

Ejemplo tabla LL en código:

```
tabla = {
  "E": {
    "num": ["T", "E'"],
    "(": ["T", "E'"]
  },
  "E'": {
    "+": ["+", "T", "E'"],
    ")": ["ε"],
    "$": ["ε"]
  },
  "T": {
    "num": ["F", "T'"],
    "(": ["F", "T'"]
  },
  "T'": {
    "*": ["*", "F", "T'"],
    "+": ["ε"],
    ")": ["ε"],
    "$": ["ε"]
  },
  "F": {
    "num": ["num"],
    "(": ["(", "E", ")"]
  }
}
```

Entrada válida:

num + num * num $

Salida esperada:

```
Pila: ['$', 'E'], Entrada: ['num', '+', 'num', '*', 'num', '$']
Aplicar: E → ['T', "E'"]


Pila: ['$', "E'", 'T'], Entrada: ['num', '+', 'num', '*', 'num', '$']
Aplicar: T → ['F', "T'"]


Pila: ['$', "E'", "T'", 'F'], Entrada: ['num', '+', 'num', '*', 'num', '$']
Aplicar: F → ['num']


Pila: ['$', "E'", "T'", 'num'], Entrada: ['num', '+', 'num', '*', 'num', '$']
Match: num
Pila: ['$', "E'", "T'"], Entrada: ['+', 'num', '*', 'num', '$']
Aplicar: T' → ['ε']


Pila: ['$', "E'"], Entrada: ['+', 'num', '*', 'num', '$']
Aplicar: E' → ['+', 'T', "E'"]


Pila: ['$', "E'", 'T', '+'], Entrada: ['+', 'num', '*', 'num', '$']
Match: +
Pila: ['$', "E'", 'T'], Entrada: ['num', '*', 'num', '$']
Aplicar: T → ['F', "T'"]


Pila: ['$', "E'", "T'", 'F'], Entrada: ['num', '*', 'num', '$']
Aplicar: F → ['num']


Pila: ['$', "E'", "T'", 'num'], Entrada: ['num', '*', 'num', '$']
Match: num
Pila: ['$', "E'", "T'"], Entrada: ['*', 'num', '$']
Aplicar: T' → ['*', 'F', "T'"]


Pila: ['$', "E'", "T'", 'F', '*'], Entrada: ['*', 'num', '$']
Match: *
Pila: ['$', "E'", "T'", 'F'], Entrada: ['num', '$']
Aplicar: F → ['num']


Pila: ['$', "E'", "T'", 'num'], Entrada: ['num', '$']
Match: num
Pila: ['$', "E'", "T'"], Entrada: ['$']
Aplicar: T' → ['ε']


Pila: ['$', "E'"], Entrada: ['$']
Aplicar: E' → ['ε']


Pila: ['$'], Entrada: ['$']
Match: $
```

Ejemplo 3.
Gramática:

```
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → ( E ) | num
```

Entrada inválida:
num + * num

Salida esperada:
Error sintáctico

Ejemplo 4.
Gramática:

```
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → ( E ) | num
```

Entrada inválida:
((num + num) *

Salida esperada:
Error sintáctico

5. Entregables

1. Código fuente del parser.
2. Documento breve que explique:
    a. Cómo funciona su implementación.
    b. Cómo ejecutar el programa.
    c. Descripción de las pruebas realizadas, incluyendo los 4 ejemplos anteriormente descritos y las respectivas salidas generadas por el parser.

- **Ejercicio <u>individual.</u>**
- **Entrega:** Martes 10 de Noviembre **<u>(antes de la clase)</u>**.

- Enviar entregables por correo electrónico a la dirección melissa.brigitthe.hinojosa@ingenieria.unam.edu.
- Presentación breve (del código en ejecución) en clase.

Rúbrica para evaluación de ejercicios prácticos y tareas

Calificación	Criterio
Excelente	1. Corrección técnica. El programa funciona correctamente para todos los casos requeridos y maneja errores adecuadamente. 2. Diseño y estructura del código. Código modular, claro, organizado, con funciones/clases bien definidas. 3. Correcta implementación del modelo teórico. Refleja correctamente la teoría vista en clase. 4. Manejo de casos de prueba y validación. Incluye casos de prueba adecuados y demuestra verificación. 5. Documentación y claridad. Comentarios adecuados, README (o documento PDF) claro, explicación breve del funcionamiento.
Bueno	1. Corrección técnica. Funciona en la mayoría de los casos, con fallos menores. 2. Diseño y estructura del código. Estructura adecuada con algunos problemas menores de organización. 3. Correcta implementación del modelo teórico. Existen pequeñas imprecisiones formales. 4. Manejo de casos de prueba y validación. Incluye algunos casos de prueba pero no son exhaustivos. 5. Documentación y claridad. Documentación suficiente pero mejorable.
Suficiente	1. Corrección técnica. Presenta errores funcionales pero muestra comprensión parcial. 2. Diseño y estructura del código. Código poco modular o con estructura confusa. 3. Correcta implementación del modelo teórico. Existen errores conceptuales leves. 4. Manejo de casos de prueba y validación. Se realizaron pruebas mínimas.

Calificación	Criterio
	5. Documentación y claridad. Incluye comentarios mínimos.
Insuficiente	1. Corrección técnica. No funciona o no cumple con lo solicitado. 2. Diseño y estructura del código. Código desorganizado o difícil de entender. 3. Correcta implementación del modelo teórico. Existen errores graves en la aplicación teórica. 4. Manejo de casos de prueba y validación. Sin pruebas o sin evidencia de validación. 5. Documentación y claridad. Sin documentación.